Least Squares

Assignment

TTK4260

Instructions: Solve using python notebook.

Question 1 *(Linear Model Setup and Normal Equations)*

A renewable energy company is testing solar panel efficiency. They collected power output data at different irradiance levels:

Irradiance x (W/m^2)	Power Output y (W)
100	18.2
200	36.8
300	54.1
400	73.5
500	91.2

Assume the model $y = \theta_0 + \theta_1 x + \varepsilon$.

(a) Set up the design matrix $\mathbf{X}$ and response vector $\mathbf{y}$ for this problem. *[4 points]*

(b) Calculate $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{y}$ by hand. *[6 points]*

(c) Solve the normal equations to find $\hat{\boldsymbol{\theta}}$ and interpret the physical meaning of both parameters. *[8 points]*

(d) Calculate the residuals and comment on the model fit quality. *[6 points]*

Question 2 *(Feature Engineering and Polynomial Models)*

A materials engineer is studying the stress-strain relationship for a new polymer. The data suggests a quadratic relationship:

Strain x (%)	Stress y (MPa)
0.5	12.1
1.0	22.8
1.5	31.2
2.0	37.5
2.5	41.8
3.0	44.2

The engineer wants to fit $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \varepsilon$.

(a) Construct the appropriate design matrix for this polynomial model. *[6 points]*

(b) Explain why this is still considered a "linear" model from a least squares perspective. *[4 points]*

(c) Set up the normal equations and solve for the parameter estimates. *[10 points]*

(d) Compare the quadratic fit to a simple linear model. Which is better and why? *[6 points]*

Question 3 *(Gradient Descent Implementation)*

Consider the simple least squares problem with cost function $J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(y_i - \theta_0 - \theta_1 x_i)^2$ for the dataset: $(x_i, y_i) = \{(1,3),(2,5),(3,7),(4,9)\}$.

(a) Derive the partial derivatives $\frac{\partial J}{\partial \theta_0}$ and $\frac{\partial J}{\partial \theta_1}$. *[6 points]*

(b) Write the batch gradient descent update equations. *[4 points]*

(c) Implement gradient descent in Python with learning rate $\alpha = 0.01$ and initial guess $\theta_0 = 0, \theta_1 = 0$. Run for 1000 iterations. *[8 points]*

(d) Plot the cost function over iterations and explain the convergence behavior. *[6 points]*

(e) Compare your result with the analytical solution using normal equations. *[4 points]*

Question 4 *(Brute Force Optimization and Parameter Landscapes)*

For the cost function $J(\theta_1, \theta_2) = (\theta_1 - 2)^2 + 10(\theta_2 - \theta_1^2)^2$ (similar to the examples in the lecture):

(a) Implement a brute force grid search over the ranges $\theta_1 \in [-2, 4]$ and $\theta_2 \in [-1, 5]$ with step size 0.1. *[8 points]*

(b) Create a contour plot of the cost function and mark the minimum found by grid search. *[6 points]*

(c) Compare the computational time of grid search vs gradient descent for this problem. *[4 points]*

(d) Discuss when brute force might be preferable despite its computational cost. *[4 points]*

Question 5 *(Non-linear Least Squares (BOD Model))*

Environmental engineers use the Biochemical Oxygen Demand (BOD) model to study water quality. The model is $y = \theta_1(1 - e^{-\theta_2 x})$ where y is oxygen demand and x is incubation time. Given data:

Time x (days)	BOD y (mg/L)
1	10.6
2	16.0
3	18.8
4	20.9
5	22.2
7	23.8

(a) Explain why standard linear least squares (normal equations) cannot be directly applied to this model. *[5 points]*

(b) Implement parameter estimation using `scipy.optimize.minimize` with initial guess $\theta_1 = 25, \theta_2 = 0.5$. *[10 points]*

(c) Create a cost function landscape plot for $\theta_1 \in [20, 30]$ and $\theta_2 \in [0.2, 0.8]$ to visualize the optimization problem. *[8 points]*

(d) Validate your fitted model by plotting predicted vs observed values and calculating residuals. *[6 points]*

Question 6 *(Scaling Effects and Numerical Issues)*

A process control engineer has temperature data with very different scales:

Time x (seconds)	Temperature y (K)
3600	298.2
7200	423.7
10800	548.1
14400	671.8
18000	798.5

(a) Fit the model $y = \theta_0 + \theta_1 x$ using both the original data and scaled data. *[8 points]*

(b) Implement gradient descent for both cases with the same learning rate. What happens and why? *[8 points]*

(c) Explain when feature scaling is critical and when it might not be necessary. *[4 points]*

Question 7 *(Weighted Least Squares Application)*

A sensor network monitors air quality, but different sensors have different precision levels. The data shows PM2.5 measurements with associated uncertainties:

Hour x	PM2.5 y (μg/m^3)	Std Error σ_i
6	45.2	2.1
9	52.8	1.5
12	67.3	3.2
15	71.5	1.8
18	58.9	2.5
21	41.7	1.2

(a) Set up the weighted least squares problem with weights $w_i = 1/\sigma_i^2$. Justify this choice. *[6 points]*

(b) Derive and implement the weighted normal equations: $\mathbf{X}^T\mathbf{W}\mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{X}^T\mathbf{W}\mathbf{y}$. *[8 points]*

(c) Compare the WLS solution with ordinary least squares. How do the parameter estimates and fitted values differ? *[8 points]*

(d) Calculate and compare the weighted and unweighted residuals. Which approach gives a better representation of measurement uncertainty? *[6 points]*

Question 8 *(Comprehensive Python Implementation)*

Design a complete least squares analysis pipeline for a real engineering problem of your choice. Your solution should include:

(a) Problem statement with realistic data (minimum 8 data points). *[5 points]*

(b) Implementation of at least three solution methods: normal equations, gradient descent, and scipy optimization. *[12 points]*

(c) Comprehensive residual analysis including plots and statistical measures. *[8 points]*

(d) Feature engineering demonstration (e.g., polynomial terms, transformations). *[6 points]*

(e) Model validation and discussion of assumptions. *[7 points]*

(f) Professional-quality visualization of results with appropriate labels and legends. *[6 points]*

(g) Critical discussion of when each method would be preferred in practice. *[6 points]*